Bernoulli Maps: A Universal Framework for Approximate Computation

Alexander Towell
atowell@siue.edu

July 29, 2025

Abstract

We introduce Bernoulli maps as a general framework for understanding computation through the distinction between latent functions (the true mathematical mappings) and their observations (the computed approximations). Every practical algorithm observes a latent function through a noisy channel—whether due to randomization, resource bounds, or intentional approximation. We show that randomized algorithms, sketching techniques, hash functions, and lossy compression are unified by this perspective: they all implement observed functions that approximate latent mathematical functions with controlled error. By making the latent/observed distinction explicit at the type level, we enable compositional reasoning about error propagation and provide a foundation for designing algorithms with predictable approximation characteristics. The framework reveals that approximation is not a compromise but a fundamental aspect of bridging mathematical functions with computational reality.

1 Introduction

1.1 The Latent/Observed Duality in Computation

In mathematics, functions are perfect mappings from inputs to outputs. In computation, we can only *observe* these latent functions through the lens of finite resources, randomization, and approximation. This fundamental gap between latent mathematical functions and their observed computational implementations appears everywhere:

- **Randomized algorithms** observe latent deterministic functions through random choices
- **Lossy compression** observes latent data through bounded representations
- **Numerical methods** observe latent real functions through floating-point arithmetic
- **Hash functions** observe latent bijections through finite-range mappings
- **Machine learning** observes latent patterns through learned approximations

We propose *Bernoulli maps* as a unifying framework that makes this latent/observed distinction explicit. A Bernoulli map is an observed function that approximates a latent function with controlled error probabilities—capturing the reality that computation is fundamentally about observing the unobservable.

1.2 Motivating Example: Primality Testing

Consider the problem of testing whether a number is prime. The latent function $\mathsf{isPrime} : \mathbb{N} \to \texttt{Bool}$ is well-defined mathematically but expensive to compute exactly. The Miller-Rabin test [4, 6] provides an *observation* of this latent function in $O(k \log^3 n)$ time.

We model this as observing the latent function through a noisy channel:

- **Latent**: $\mathsf{isPrime}(n)$ - the true primality status
- **Observed**: $\widetilde{\mathsf{isPrime}}_k(n)$ - Miller-Rabin's output after k rounds

The observation quality is characterized by:

$$\mathbb{P}\left[\widetilde{\mathsf{isPrime}}_k(n) = \texttt{true} \mid \mathsf{isPrime}(n) = \texttt{true}\right] = 1 \quad \text{(no false negatives)} \tag{1}$$

$$\mathbb{P}\left[\widetilde{\mathsf{isPrime}}_k(n) = \texttt{true} \mid \mathsf{isPrime}(n) = \texttt{false}\right] \leq 4^{-k} \quad \text{(controlled false positives)} \tag{2}$$

This exemplifies the key principle: we observe latent mathematical truth through practical computation.

2 Theory of Bernoulli Maps

2.1 Basic Definitions

Definition 2.1 (Bernoulli Map)**.** *A Bernoulli map models the observation of a latent function through a probabilistic channel. Given a latent function $f : X \to Y$, an observed function $\widetilde{f} : X \to Y$ is characterized by the conditional distribution:*

$$\mathbb{P}\left[\widetilde{f}(x) = y \mid f(x) = y'\right] = p_{x,y',y} \tag{3}$$

This captures how we observe the latent output $f(x) = y'$ as the potentially different value y.

Definition 2.2 (Error Profile)**.** *The error profile quantifies the fidelity of observations. For latent f and observed $\widetilde{f}$, the error at input x is:*

$$\epsilon(x) = \mathbb{P}\left[\textit{observe } \widetilde{f}(x) \neq y \mid \textit{latent } f(x) = y\right] \tag{4}$$

This measures how often our observation differs from the latent truth.

Remark 2.3 (Interpretation)**.** *Every algorithm that computes a function is effectively providing observations of a latent mathematical function. The error profile characterizes the quality of these observations—whether errors arise from randomization, truncation, or resource bounds.*

2.2 Confusion Matrix for Functions

When the domain and codomain are finite, we can represent a Bernoulli map using a confusion matrix, generalizing the concept from classification:

Definition 2.4 (Function Confusion Matrix)**.** *For functions of type $X \mapsto Y$ where $|X \mapsto Y| = n$, the confusion matrix $Q \in \mathbb{R}^{n \times n}$ has entries:*

$$q_{ij} = \mathbb{P}\left[\textit{observe } f_j | \textit{latent } f_i\right] \tag{5}$$

where $f_1, \ldots, f_n$ enumerate all functions from X to Y.

Example 2.5 (Boolean Function Confusion Matrix)**.** *For* $\texttt{Bool} \mapsto \texttt{Bool}$*, there are 4 functions:* id*,* not*,* true*,* false*. A first-order Bernoulli approximation has confusion matrix:*

latent \ *observed*	id	not	true	false
id	$1-\epsilon$	$\epsilon/3$	$\epsilon/3$	$\epsilon/3$
not	$\epsilon/3$	$1-\epsilon$	$\epsilon/3$	$\epsilon/3$
true	$\epsilon/3$	$\epsilon/3$	$1-\epsilon$	$\epsilon/3$
false	$\epsilon/3$	$\epsilon/3$	$\epsilon/3$	$1-\epsilon$

This represents maximum entropy distribution over errors.

Proposition 2.6 (Function Confusion Matrix Properties)**.**

1. *The order (degrees of freedom) is at most* $n(n-1)$ *where* $n = |Y|^{|X|}$
2. *Function equality probability:* $\mathbb{P}\left[\widetilde{f} = f\right] = q_{ff}$
3. *Pointwise error accumulates:* $\mathbb{P}\left[\widetilde{f} = f\right] = \prod_{x \in X} \mathbb{P}\left[\widetilde{f}(x) = f(x)\right]$ *for independent errors*

2.3 Orders of Approximation

Like Bernoulli sets, Bernoulli maps have different orders based on their error structure:

Definition 2.7 (First-Order Bernoulli Map)**.** *A first-order Bernoulli map has uniform error rate:*

$$\mathbb{P}\left[\widetilde{f}(x) \neq f(x)\right] = \epsilon \quad \forall x \in \mathsf{dom}(f) \tag{6}$$

Definition 2.8 (Second-Order Bernoulli Map)**.** *A second-order Bernoulli map distinguishes between false positives and false negatives:*

$$\mathbb{P}\left[\widetilde{f}(x) \neq \bot | f(x) = \bot\right] = \alpha \tag{7}$$

$$\mathbb{P}\left[\widetilde{f}(x) = \bot | f(x) \neq \bot\right] = \beta \tag{8}$$

where $\bot$ *represents undefined.*

2.4 Partial Functions

Many real-world functions are partial. A partial function $f : X \rightharpoonup Y$ can be modeled as a total function $f : X \mapsto \texttt{Maybe}[Y]$.

Theorem 2.9 (Partial Function Approximation)**.** *For a partial function* f^* *with domain of definition* $\mathsf{dom}(\mathsf{f}^*)$*, a Bernoulli approximation* $\widetilde{\mathsf{f}}^*$ *has:*

$$\mathsf{dom}(\widetilde{\mathsf{f}}^*) = \widetilde{\mathsf{dom}(\mathsf{f}^*)}[\alpha][\beta] \tag{9}$$

where $\widetilde{\cdot}$ *denotes a Bernoulli set.*

3 Composition of Bernoulli Maps

3.1 Sequential Composition

Theorem 3.1 (Error Propagation in Composition)**.** *For Bernoulli maps $\widetilde{\mathsf{f}} : X \mapsto Y$ and $\widetilde{\mathsf{g}} : Y \mapsto Z$ with error rates ϵ_f and ϵ_g, the composition $\widetilde{\mathsf{g}} \circ \widetilde{\mathsf{f}}$ has error rate bounded by:*

$$\epsilon_{g \circ f}(x) \leq \epsilon_f(x) + \epsilon_g(\mathsf{f}(x)) - \epsilon_f(x) \cdot \epsilon_g(\mathsf{f}(x)) \tag{10}$$

Proof. The probability of correct computation is:

$$\mathbb{P}\left[(\widetilde{\mathsf{g}} \circ \widetilde{\mathsf{f}})(x) = (\mathsf{g} \circ \mathsf{f})(x)\right] = \mathbb{P}\left[\widetilde{\mathsf{f}}(x) = \mathsf{f}(x) \wedge \widetilde{\mathsf{g}}(\mathsf{f}(x)) = \mathsf{g}(\mathsf{f}(x))\right] \tag{11}$$

$$\geq (1 - \epsilon_f(x))(1 - \epsilon_g(\mathsf{f}(x))) \tag{12}$$

□

3.2 Higher-Order Bernoulli Maps

Composition naturally produces higher-order approximations:

Definition 3.2 (Order of Composition)**.** *The composition of k first-order Bernoulli maps produces a map of order at most $2^k - 1$, as different error paths create distinct distributions.*

Remark 3.3 (Non-uniform Error Distribution)**.** *A critical observation: composition destroys the independence assumption of first-order approximations. Elements that pass through different "error paths" in the composition have different error probabilities, making the output errors non-identically distributed. This fundamentally changes the statistical properties and complicates analysis.*

Example 3.4 (Iterated Approximation)**.** *Consider iterating a Bernoulli map $\widetilde{\mathsf{f}} : X \to X$:*

- $\widetilde{\mathsf{f}}^1$*: First-order (uniform errors)*
- $\widetilde{\mathsf{f}}^2 = \widetilde{\mathsf{f}} \circ \widetilde{\mathsf{f}}$*: Up to third-order*
- $\widetilde{\mathsf{f}}^n$*: Exponentially many error classes*

3.3 Algebraic Operations on Bernoulli Maps

For set-indicator functions, we can define algebraic operations:

Theorem 3.5 (Set Operation Error Rates)**.** *For Bernoulli approximations of indicator functions $\mathbf{1}_A$ and $\mathbf{1}_B$:*

$$FPR(\mathbf{1}_{A \cap B}) = FPR(A) \cdot FPR(B) \tag{13}$$

$$FNR(\mathbf{1}_{A \cap B}) = 1 - (1 - FNR(A))(1 - FNR(B)) \tag{14}$$

$$FPR(\mathbf{1}_{A \cup B}) = 1 - (1 - FPR(A))(1 - FPR(B)) \tag{15}$$

$$FNR(\mathbf{1}_{A \cup B}) = FNR(A) \cdot FNR(B) \tag{16}$$

under independence assumptions.

Proof. These follow from viewing set operations as Boolean operations on indicator functions and applying error propagation rules for AND/OR operations. □

3.4 Parallel Composition

Definition 3.6 (Product of Bernoulli Maps)**.** *Given* $\widetilde{\mathsf{f}} : X \mapsto Y$ *and* $\widetilde{\mathsf{g}} : X \mapsto Z$*, the product* $\widetilde{\mathsf{f}} \times \widetilde{\mathsf{g}} : X \mapsto Y \times Z$ *is defined by:*

$$(\widetilde{\mathsf{f}} \times \widetilde{\mathsf{g}})(x) = (\widetilde{\mathsf{f}}(x), \widetilde{\mathsf{g}}(x)) \tag{17}$$

4 Case Studies

4.1 Primality Testing

The Miller-Rabin primality test demonstrates a Bernoulli map with one-sided error:

Input: $n > 2$, odd integer; k iterations
Output: `true` if n is probably prime, `false` if composite
Write $n - 1 = 2^r \cdot d$ with d odd;
for $i = 1$ *to* k **do**
  Choose random $a \in [2, n-2]$;
  $x \leftarrow a^d \bmod n$;
  if $x = 1$ *or* $x = n - 1$ **then**
    continue to next witness
  end
  for $j = 1$ *to* $r - 1$ **do**
    $x \leftarrow x^2 \bmod n$;
    if $x = n - 1$ **then**
      continue to next witness
    end
  end
  return `false`;
end
return `true`;

Algorithm 1: Miller-Rabin Primality Test

Theorem 4.1 (Miller-Rabin Error Rate)**.** *For composite* n*, the Miller-Rabin test with* k *iterations has:*

$$\mathbb{P}\left[\widetilde{\mathsf{isPrime}}_k(n) = \texttt{true} | n \text{ is composite}\right] \leq 4^{-k} \tag{18}$$

4.2 Hash Functions

Hash functions exemplify the latent/observed paradigm: the latent identity function (perfect discrimination of inputs) is observed through a finite-range mapping.

Definition 4.2 (Hash Function as Observation)**.** *A hash function* $h : X \to [m]$ *observes the latent injection* $\mathsf{id} : X \hookrightarrow \mathbb{N}$*:*

- ***Latent****: Each* $x \in X$ *maps to a unique natural number*
- ***Observed****:* $h(x) \in [m]$ *provides a finite-precision observation*

The observation quality for discrimination is:

$$\mathbb{P}\left[\widetilde{(x \neq y)} \mid \text{latent } x \neq y\right] = \mathbb{P}\left[h(x) \neq h(y)\right] = 1 - \frac{1}{m} \tag{19}$$

Remark 4.3. *Hash functions sacrifice perfect injectivity (latent) for finite representation (observed), enabling practical implementations of sets, maps, and caches.*

4.3 Sketching Algorithms

Sketching algorithms observe latent frequency distributions through space-bounded data structures:

Definition 4.4 (Count-Min Sketch as Observation)**.** *The Count-Min sketch [1] observes the latent frequency function* $\mathsf{freq} : X \to \mathbb{N}$*:*

- ***Latent****: True frequency* $\mathsf{freq}(x)$ *for each element*
- ***Observed****: Sketch query* $\widetilde{\mathsf{freq}}_{\epsilon,\delta}(x)$ *using* $O(\frac{1}{\epsilon} \log \frac{1}{\delta})$ *space*

The observation guarantee:

$$\mathbb{P}\left[\widetilde{\mathsf{freq}}(x) \geq \mathsf{freq}(x) \mid \textit{any input}\right] = 1 \quad \textit{(no underestimation)} \tag{20}$$

$$\mathbb{P}\left[\widetilde{\mathsf{freq}}(x) > \mathsf{freq}(x) + \epsilon\|\mathbf{f}\|_1 \mid \textit{stream } \mathbf{f}\right] < \delta \tag{21}$$

This shows how we trade space for observation quality: perfect observation would require $O(|X|)$ space, but we achieve useful observations with logarithmic space.

4.4 Monte Carlo Integration

Monte Carlo methods approximate integrals through random sampling:

Example 4.5 (Monte Carlo Integration)**.** *To approximate* $I = \int_a^b f(x)dx$*, we use:*

$$\widetilde{\mathsf{I}}_n = \frac{b-a}{n} \sum_{i=1}^{n} f(X_i) \tag{22}$$

where X_i *are uniform random samples. This is a Bernoulli map with:*

$$\mathbb{E}\left[\left|\widetilde{\mathsf{I}}_n - I\right|^2\right] = \frac{(b-a)^2\, Var(f)}{n} \tag{23}$$

5 Computational Trade-offs

5.1 Time-Accuracy Trade-off

Many algorithms exhibit a fundamental trade-off between computation time and accuracy:

Theorem 5.1 (Time-Accuracy Trade-off)**.** *For a class of problems with inherent complexity* C*, any algorithm running in time* $T < C$ *must be a Bernoulli map with error rate:*

$$\epsilon \geq 1 - \frac{T}{C} \tag{24}$$

5.2 Space-Accuracy Trade-off

Similarly, space-bounded computation induces approximation:

Theorem 5.2 (Space-Accuracy Trade-off)**.** *Any algorithm using space* S *to represent a set of size* $N > 2^S$ *must have error rate:*

$$\epsilon \geq 1 - \frac{2^S}{N} \tag{25}$$

5.3 Communication-Accuracy Trade-off

In distributed systems, limited communication induces approximation:

Example 5.3 (Distributed Mean Estimation)**.** *To compute the mean of n values distributed across k nodes with B bits of communication per node:*

$$\epsilon \geq \frac{\sigma^2}{n} \cdot \frac{k}{2^{2B/k}} \tag{26}$$

where σ^2 is the variance.

6 Codec Theory for Bernoulli Maps

6.1 Universal Construction

We can systematically construct Bernoulli maps through coding theory:

Theorem 6.1 (Universal Bernoulli Map Construction)**.** *Given any function $\mathsf{f} : X \mapsto Y$ and error profile ϵ, there exists a Bernoulli map $\widetilde{\mathsf{f}}$ achieving this profile using:*

$$H(\epsilon) + \log |Y| \text{ bits per input} \tag{27}$$

where H is the binary entropy function.

6.2 Optimal Encoding

Definition 6.2 (Rate-Distortion Function)**.** *For a function f and distortion measure d, the rate-distortion function is:*

$$R(D) = \min_{p(\hat{y}|x)} I(X; \hat{Y}) \tag{28}$$

subject to $\mathbb{E}\left[d(\mathsf{f}(X), \hat{Y})\right] \leq D$.

6.3 Singular Hash Map: Theoretical Optimality

We present a theoretical construction achieving information-theoretic bounds:

Definition 6.3 (Singular Hash Map)**.** *Given a partial function $\mathsf{f} : X \rightharpoonup Y$ and random oracle $h : X \times \mathbb{N} \to \{0,1\}^\infty$, the singular hash map:*

1. *Finds seed s such that $\forall x \in dom(f) : h(x,s)|_n = encode(f(x))$*
2. *Stores only the seed s*
3. *Evaluates $\widetilde{\mathsf{f}}(x) = decode(h(x,s)|_n)$*

where $|_n$ denotes taking the first n bits.

Theorem 6.4 (Singular Hash Map Properties)**.** *For false positive rate $\alpha = 2^{-k}$ and $m = |dom(f)|$:*

- *Space: $m(k + \log |Y|)$ bits (information-theoretic optimal)*
- *Construction time: $O(2^{m(k+\log |Y|)})$ expected*

- *Query time:* $O(1)$
- *False negative rate: 0*

Proof. The probability that a random seed satisfies all constraints is:

$$p = 2^{-m(k+\log|Y|)} \tag{29}$$

Thus expected trials until success is $1/p$. The seed itself requires $\log(1/p) = m(k + \log|Y|)$ bits, achieving the entropy bound $H(\alpha) + \log|Y|$ per element. □

Remark 6.5 (Practical Considerations)**.** *While theoretically optimal, the exponential construction time makes this impractical. However, it:*

- *Proves achievability of the bound*
- *Suggests relaxations (e.g., allow small false negative rate)*
- *Motivates approximate algorithms targeting this bound*

7 Oblivious Computation

Bernoulli maps enable privacy-preserving computation:

Definition 7.1 (Oblivious Bernoulli Map)**.** *A Bernoulli map* $\widetilde{\mathsf{f}}$ *is* ϵ*-oblivious if for all* $x, x' \in X$*:*

$$\frac{\mathbb{P}\left[\widetilde{\mathsf{f}}(x) = y\right]}{\mathbb{P}\left[\widetilde{\mathsf{f}}(x') = y\right]} \leq e^{\epsilon} \tag{30}$$

Theorem 7.2 (Composition of Oblivious Maps)**.** *The composition of* ϵ_1*-oblivious and* ϵ_2*-oblivious Bernoulli maps is* $(\epsilon_1 + \epsilon_2)$*-oblivious.*

7.1 Function Equality Testing

A fundamental problem is determining whether two functions are equal. The latent truth "$f = g$" is often unobservable directly—we can only observe their behavior on a finite sample of inputs.

Definition 7.3 (Observing Function Equality)**.** *Given latent functions* $f, g : X \to Y$*, we observe their equality through sampling:*

$$\widetilde{(f = g)}_n := \bigwedge_{i=1}^{n} (f(x_i) = g(x_i)) \tag{31}$$

where $x_1, \ldots, x_n$ *are sampled uniformly from* X*. This provides an observation of the latent equality through* n *point evaluations.*

Remark 7.4. *This exemplifies the latent/observed principle: the latent mathematical fact "*$f = g$*" can only be observed through finite computation, introducing the possibility of false conclusions.*

Theorem 7.5 (Function Equality Error Bounds)**.** *If $f \neq g$ with k points of disagreement in X, then:*

$$\mathbb{P}\left[f \hat{=} g \text{ returns true}\right] = \left(1 - \frac{k}{|X|}\right)^n \tag{32}$$

Thus the false positive rate is bounded:

$$\epsilon \in \left[\left(1 - \frac{1}{|X|}\right)^n, \left(\frac{n}{|X|}\right)^n\right] \tag{33}$$

Example 7.6 (Testing Hash Function Quality)**.** *To verify that a hash function $h : \{0,1\}^{64} \mapsto \{0,1\}^{32}$ behaves randomly:*

```
bool appears_random(hash_function h, int samples = 10000) {
    // Test uniform distribution
    map<uint32_t, int> counts;
    for (int i = 0; i < samples; i++) {
        uint64_t x = random_uint64();
        counts[h(x)]++;
    }

    // Chi-squared test for uniformity
    double chi_squared = compute_chi_squared(counts, samples);
    return chi_squared < critical_value(0.05);
}
```

This is a Bernoulli approximation of the ideal test that would check all 2^{64} inputs.

8 Implementation Framework

8.1 Generic Programming

We can implement Bernoulli maps generically in C++:

```
template <typename X, typename Y>
class bernoulli_map {
public:
    using input_type = X;
    using output_type = Y;
    using error_profile = std::function<double(const X&)>;

    virtual Y operator()(const X& x) const = 0;
    virtual double error_rate(const X& x) const = 0;
};

template <typename X, typename Y, typename Z>
class composed_bernoulli_map : public bernoulli_map<X, Z> {
    bernoulli_map<X, Y> f;
    bernoulli_map<Y, Z> g;
public:
```

```
    Z operator()(const X& x) const override {
        return g(f(x));
    }
    double error_rate(const X& x) const override {
        // Error propagation formula
        double e_f = f.error_rate(x);
        double e_g = g.error_rate(f(x));
        return e_f + e_g - e_f * e_g;
    }
};
```

8.2 Performance Optimizations

Key optimizations for Bernoulli maps include:

- **Memoization**: Cache results for deterministic approximations
- **Adaptive precision**: Adjust error rates based on context
- **Vectorization**: Process multiple inputs simultaneously
- **GPU acceleration**: Parallelize Monte Carlo sampling

9 Applications

9.1 Database Query Optimization

Approximate query processing uses Bernoulli maps to trade accuracy for speed:

Example 9.1 (Approximate COUNT DISTINCT)**.** *Instead of exact counting, use HyperLogLog:*

$$\widetilde{\mathsf{count}}_m(S) = \frac{\alpha_m m^2}{\sum_{j=1}^{m} 2^{-M[j]}} \tag{34}$$

with relative error $\approx 1.04/\sqrt{m}$*.*

9.2 Network Function Virtualization

Network functions can be approximated for performance:

Example 9.2 (Approximate Packet Classification)**.** *Replace exact matching with probabilistic filters:*

$$\mathbb{P}\left[\widetilde{\mathsf{classify}}(p) = \mathsf{classify}(p)\right] \geq 1 - \delta \tag{35}$$

achieving $O(1)$ *classification time.*

9.3 Machine Learning

Neural networks are Bernoulli maps with learned error profiles:

Example 9.3 (Neural Network as Bernoulli Map)**.** *A neural network* $\widetilde{\mathsf{NN}}_\theta : \mathbb{R}^n \mapsto [k]$ *approximates the true labeling function with:*

$$\epsilon(x) = \mathbb{P}\left[\widetilde{\mathsf{NN}}_\theta(x) \neq \mathsf{label}(x)\right] \tag{36}$$

minimized during training.

10 Extension to Relations

10.1 Approximate Relations

While functions are special cases of relations, the framework extends naturally to arbitrary relations:

Definition 10.1 (Observed k-ary Relation)**.** *Given a latent k-ary relation $R \subseteq X_1 \times \cdots \times X_k$, an observed relation $\widetilde{R}$ is a k-ary predicate:*

$$\widetilde{R} : X_1 \times \cdots \times X_k \to \{0, 1\} \tag{37}$$

with error rates α (false positive) and β (false negative) for tuple membership.

Example 10.2 (Approximate Ordering)**.** *For a partial order $\leq$ on set X, the observed relation $\widetilde{\leq}$ satisfies:*

- *If $x \leq y$: $\mathbb{P}\left[[]\, \widetilde{\leq}(x, y) = 1\right] = 1 - \beta$*
- *If $x \not\leq y$: $\mathbb{P}\left[[]\, \widetilde{\leq}(x, y) = 1\right] = \alpha$*

This may violate transitivity: $\widetilde{\leq}(x, y) \wedge \widetilde{\leq}(y, z)$ need not imply $\widetilde{\leq}(x, z)$.

Proposition 10.3 (Non-uniform Error Distribution)**.** *When composing observed relations, errors become non-uniformly distributed. For relations $\widetilde{R} \subseteq X \times Y$ and $\widetilde{S} \subseteq Y \times Z$:*

$$\widetilde{S} \circ \widetilde{R} = \{(x, z) : \exists y \in Y, \widetilde{R}(x, y) \wedge \widetilde{S}(y, z)\} \tag{38}$$

The error rate for (x, z) depends on the number of intermediate elements y satisfying the relation.

11 Related Work

- **Randomized Algorithms**: Motwani and Raghavan [5] provide comprehensive coverage of randomized algorithms
- **Property Testing**: Goldreich [2] studies algorithms that approximate decision problems
- **PAC Learning**: Valiant's framework [7] for learning approximate concepts
- **Differential Privacy**: McSherry and Talwar [3] on privacy through approximation

12 Conclusions

Bernoulli maps formalize the fundamental distinction between latent mathematical functions and their computational observations. This framework reveals that every practical algorithm is observing a latent function through the constraints of finite resources, randomization, or intentional approximation.

Key insights from the latent/observed perspective:

- **Unification**: Randomized algorithms, sketches, and hash functions all observe latent functions through different channels
- **Compositionality**: Error propagation laws describe how observation quality degrades through function composition

- **Inevitability**: The gap between latent and observed is not a flaw but an inherent aspect of computation
- **Design principles**: We can systematically construct observations with desired error profiles

Future directions include:

- Quantum computing: How do quantum observations of latent functions differ?
- Synthesis: Can we automatically derive optimal observations given resource constraints?
- Hardware: Specialized circuits for common observation patterns
- Type systems: Making the latent/observed distinction explicit in programming languages
- Machine learning: Neural networks as learned observations of latent functions

The universality of Bernoulli maps demonstrates that computation is fundamentally about observing the uncomputable through the lens of the computable.

References

[1] Graham Cormode and S Muthukrishnan. An improved data stream summary: the count-min sketch and its applications. In *Journal of Algorithms*, volume 55, pages 58–75. Elsevier, 2005.

[2] Oded Goldreich. *Introduction to property testing*. Cambridge University Press, 2017.

[3] Frank McSherry and Kunal Talwar. Mechanism design via differential privacy. In *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS'07)*, pages 94–103. IEEE, 2007.

[4] Gary L Miller. Riemann's hypothesis and tests for primality. *Journal of computer and system sciences*, 13(3):300–317, 1976.

[5] Rajeev Motwani and Prabhakar Raghavan. *Randomized algorithms*. Cambridge University Press, 1995.

[6] Michael O Rabin. Probabilistic algorithm for testing primality. *Journal of number theory*, 12 (1):128–138, 1980.

[7] Leslie G Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.